

EC-231 Operating Systems

BS(CE)-2021

LAB REPORT # 07



Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz
21-CE-021	Yawar Ali Malik
21-CE-020	Muhammad Ibthesam

Department of Computer Engineering

BS Computer Engineering Program

HITEC University Taxila

	Presentation (Format (0.5) + Title (0.5) + Conclusion(1) + Procedure(2) + Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	5	5	5	15
Obtained				

Lab Instructor.
Engr. Fasih Ahmad

Experiment # 07

Implementation of File I/O Redirection System Calls in Linux- II

Objective

In this lab we will learn about system call techniques in UNIX.

Software Used

Virtual Box

Ubuntu

Procedure

1. Factorial Calculation with dup():

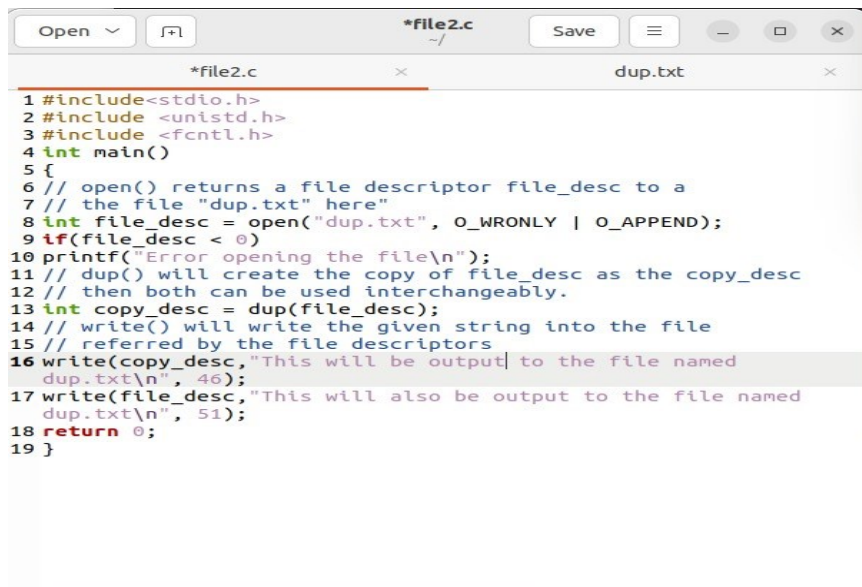
- Open a file for read and write operations.
- Duplicate the file descriptor using dup().
- Read a predefined number from the file.
- Calculate its factorial.
- Write the result back to the file.

2. Data Copying using dup2():

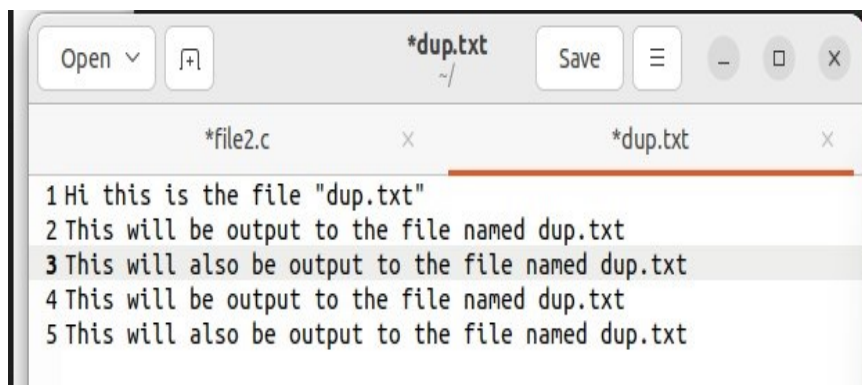
- Write a C program.
- Open source and destination files.
- Duplicate their file descriptors using dup2().
- Copy data from the source file to the destination file using read() and write() functions.
- Close the file descriptors.
- Compile and execute the program.
- Verify successful data transfer between the source and destination files.

Tasks

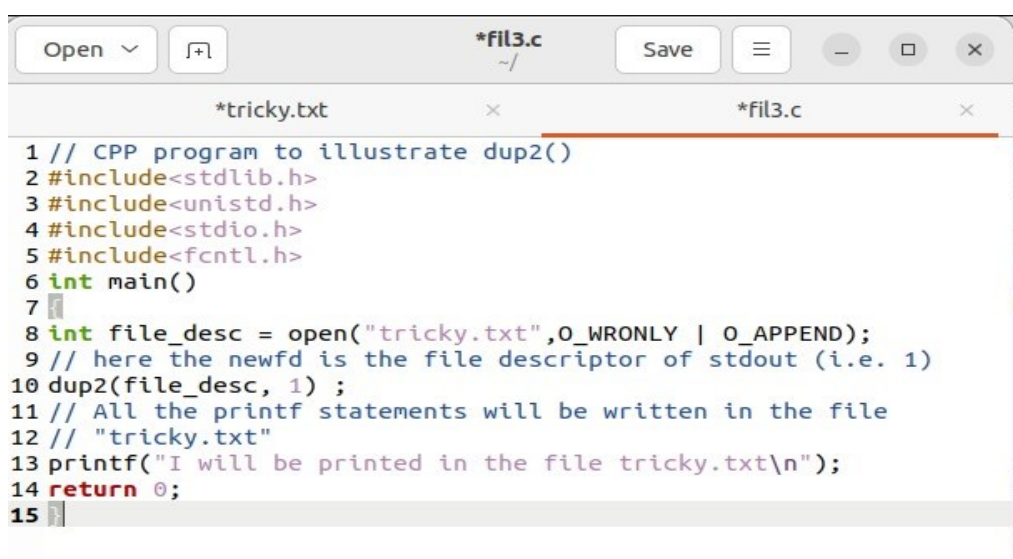
Task 1



```
1 #include<stdio.h>
2 #include <unistd.h>
3 #include <fcntl.h>
4 int main()
5 {
6 // open() returns a file descriptor file_desc to a
7 // the file "dup.txt" here"
8 int file_desc = open("dup.txt", O_WRONLY | O_APPEND);
9 if(file_desc < 0)
10 printf("Error opening the file\n");
11 // dup() will create the copy of file_desc as the copy_desc
12 // then both can be used interchangeably.
13 int copy_desc = dup(file_desc);
14 // write() will write the given string into the file
15 // referred by the file descriptors
16 write(copy_desc, "This will be output to the file named
dup.txt\n", 46);
17 write(file_desc, "This will also be output to the file named
dup.txt\n", 51);
18 return 0;
19 }
```



```
1 Hi this is the file "dup.txt"
2 This will be output to the file named dup.txt
3 This will also be output to the file named dup.txt
4 This will be output to the file named dup.txt
5 This will also be output to the file named dup.txt
```



```
1 // CPP program to illustrate dup2()
2 #include<stdlib.h>
3 #include<unistd.h>
4 #include<stdio.h>
5 #include<fcntl.h>
6 int main()
7 {
8 int file_desc = open("tricky.txt", O_WRONLY | O_APPEND);
9 // here the newfd is the file descriptor of stdout (i.e. 1)
10 dup2(file_desc, 1);
11 // All the printf statements will be written in the file
12 // "tricky.txt"
13 printf("I will be printed in the file tricky.txt\n");
14 return 0;
15 }
```

```
Open  [icon] *tricky.txt  Save  [menu]  [min]  [max]  [close]
1 I will be printed in the file tricky.txt
```

Task 2

```
Open  [icon] file2.c  Save  [menu]  [min]  [max]  [close]
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 int main() {
5     FILE *file = fopen("factorial.txt", "r+");
6
7     int number;
8     fscanf(file, "%d", &number);
9
10    int result = 1;
11    for (int i = 1; i <= number; i++) {
12        result *= i;
13    }
14    fseek(file, 0, SEEK_END);
15    dprintf(fileno(file), "Factorial is: %d\n", result);
16
17    fclose(file);
18    return 0;
19 }
```

```
Open  [icon] factorial.txt  Save  [menu]  [min]  [max]  [close]
1 7
2 Factorial is: 5040
```

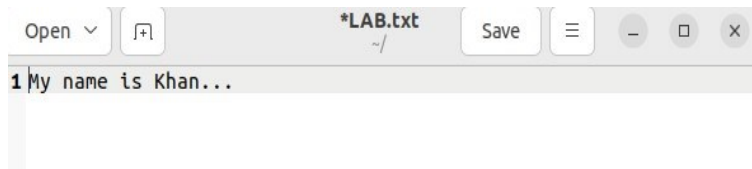
Task 3

```
Open  [icon] file1.c  Save  [menu]  [min]  [max]  [close]
1 #include <unistd.h>
2 #include <fcntl.h>
3
4 #define BUFFER_SIZE 8192
5
6 int main() {
7     int source_fd, dest_fd;
8     ssize_t bytes_read, bytes_written;
9     char buffer[BUFFER_SIZE];
10    source_fd = open("SP.txt", O_RDONLY);
11    dest_fd = open("LAB.txt", O_WRONLY | O_CREAT | O_TRUNC,
0644);
12    while ((bytes_read = read(source_fd, buffer, BUFFER_SIZE)) >
0) {
13        bytes_written = write(dest_fd, buffer, bytes_read);
14    }
15    return 0;
16 }
```

SP.txt



LAB.txt



Conclusion

In summary, by delving into the functionalities of the `dup()` and `dup2()` system calls, we have acquired the proficiency to efficiently handle file descriptors within Unix-like environments. The `dup()` function empowers us to dynamically replicate file descriptors, whereas `dup2()` further enhances this capability by enabling us to designate a preferred file descriptor number for the duplicate.